

## 02 — Case Before-Save Flow Specification

**Audience:** SF Admin, SF Developer **Purpose:** Stamp revenue tier + routing metadata on every new Case automatically at creation. **Prerequisites:** Fields from [01](#) deployed.

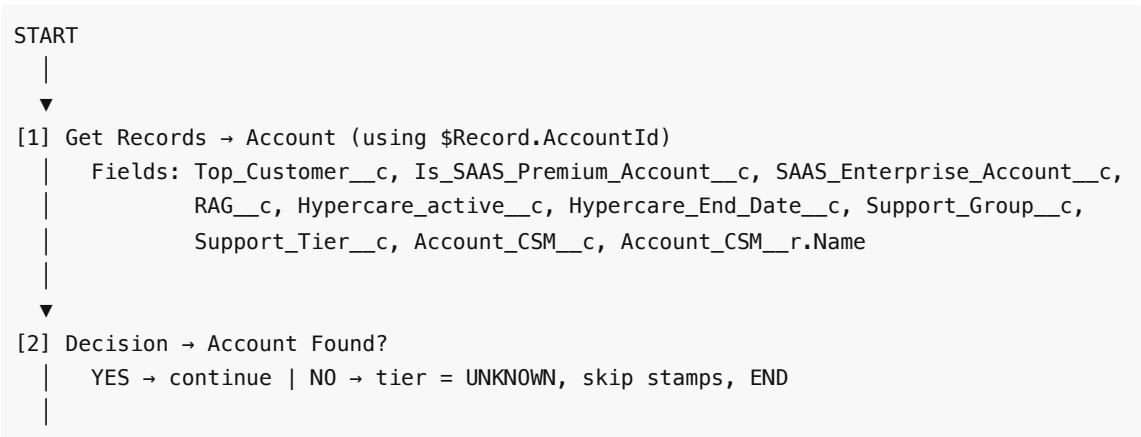
### 1. Flow Metadata

| Attribute          | Value                                                                                                                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Name           | Case_RevenueTier_BeforeSave                                                                                                                                                           |
| Label              | Case — Revenue Tier Enrichment (Before Save)                                                                                                                                          |
| Type               | Record-Triggered Flow                                                                                                                                                                 |
| Object             | Case                                                                                                                                                                                  |
| Trigger            | A record is created                                                                                                                                                                   |
| Timing             | Fast Field Updates (Before Save)                                                                                                                                                      |
| Entry Condition    | RecordType.DeveloperName IN { NormalSupport, ChangeManagement, RCA, ReleaseUpgrade, PhilippinesSupport, ProjectIssues, CognoAI, PlatformSupport, PremiumPlatformSupport, SAASAlerts } |
| Run Asynchronously | No                                                                                                                                                                                    |
| Description        | Stamps Account revenue tier, CSM, RAG, and routes Strategic/Hypercare to dedicated queues. See docs/revenue-in-support/02-case-flow-specification.md                                  |

**Why Before-Save (Fast Field Updates):** no DML needed for stamping fields on the same record, faster than After-Save, doesn't consume async governor limits. Changing OwnerId to a Queue is allowed in Before-Save.

**Why Entry Condition limits record types:** avoids running on Feedback, GDPR, legal, or partner-specific record types where tier logic doesn't apply. Adjust list based on Support Ops guidance.

### 2. Flow Canvas — Step-by-Step



```

▼
[3] Get Records → Queues (Custom Metadata Type lookup)
    |   Queue_Config__mdt by DeveloperName
    |
    ▼
[4] Decision → Which Tier? (uses Account.Support_Tier__c directly if non-null)
    |   ├── HYPERCARE → [5a]
    |   ├── STRATEGIC → [5b]
    |   ├── PREMIUM   → [5c]
    |   └── STANDARD   → [5d]
    |
    ▼
[5a/b/c/d] Assignment → stamp fields on $Record
    |   Account_Revenue_Tier__c
    |   Account_RAG_at_Creation__c
    |   Account_CSM_Name__c
    |   Is_Top_Customer__c
    |   Priority (for HYPERCARE + STRATEGIC, upgrade if Minor)
    |   Queue__c (picklist, for reporting)
    |   OwnerId (for HYPERCARE + STRATEGIC only – route to queue)
    |
    ▼
[6] (Optional) Get Records → accounts_revenue__c aggregate
    |   → Assignment → Account_LTM_Revenue__c
    |
    ▼
END

```

### 3. Element-by-Element Specification

#### [1] Get Records — Account

| Property          | Value                                                                                                                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Name          | getAccount                                                                                                                                                                                                                                                         |
| Object            | Account                                                                                                                                                                                                                                                            |
| Filter            | Id Equals {!\$Record.AccountId}                                                                                                                                                                                                                                    |
| How many records  | First record only                                                                                                                                                                                                                                                  |
| Fields to Query   | Id, Name, Top_Customer__c, Is_SAAS_Premium_Account__c, SAAS_Enterprise_Account__c, RAG__c, Hypercare_active__c, Hypercare_End_Date__c, Support_Group__c, Support_Tier__c, Account_CSM__c, Account_CSM__r.Name, Account_CSM__r.Email, Customer_Universal_Account__c |
| Store in variable | varAccount (of type Account)                                                                                                                                                                                                                                       |

**Parent vs Universal Account question:** Case.AccountId may point to a Setup/Tenant account that has its own Support\_Tier\_\_c different from the parent/Universal account. Decide per business need:

- **Option A (simplest, use child):** use `Case.AccountId` directly (recommended if tier is stamped on each tenant)
- **Option B (use parent):** walk up via `ParentId`
- **Option C (use Universal):** use `Customer_Universal_Account__c` from `accounts_revenue__c` (complex — not on `Case` natively)

**Recommendation:** Option A for v1 — tier is an Account-level concept, and if a tenant deserves different treatment than its parent, that's by design.

## [2] Decision — Account Found?

| Outcome   | Condition                              | Next                                                         |
|-----------|----------------------------------------|--------------------------------------------------------------|
| Found     | <code>{!getAccount}</code> IS NOT NULL | Continue to [3]                                              |
| Not Found | Default                                | Assign <code>Account_Revenue_Tier__c</code> = 'UNKNOWN', END |

## [3] Get Records — Queue Config (Custom Metadata Type)

**Requires:** A Custom Metadata Type `Queue_Config__mdt` with records for each tier. This avoids hardcoding Salesforce Queue IDs (which differ between sandbox and production).

**Custom Metadata Type definition:**

| Attribute | Value                                                                                                                                                       |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Name  | <code>Queue_Config__mdt</code>                                                                                                                              |
| Label     | Queue Config                                                                                                                                                |
| Fields    | <code>Tier__c</code> (Text, 20), <code>Queue_DeveloperName__c</code> (Text, 80), <code>Queue_Label__c</code> (Text, 120), <code>Active__c</code> (Checkbox) |

**Records to create:**

| DeveloperName | Tier__c   | Queue_DeveloperName__c  | Active__c |
|---------------|-----------|-------------------------|-----------|
| Hypercare     | HYPERCARE | Hypercare_Support_Queue | true      |
| Strategic     | STRATEGIC | Strategic_Support_Queue | true      |
| Premium       | PREMIUM   | Premium_Support_Queue   | true      |
| Standard      | STANDARD  | FLR_Support_Queue       | true      |

**Get Records element in Flow:**

| Property | Value                          |
|----------|--------------------------------|
| API Name | <code>getQueueConfig</code>    |
| Object   | <code>Queue_Config__mdt</code> |

|                   |                              |
|-------------------|------------------------------|
| Filter            | Active__c Equals True        |
| How many records  | All records                  |
| Store in variable | varQueueConfigs (collection) |

Then a **Loop** iterates varQueueConfigs and a **Decision** matches `varQueueConfig.Tier__c = varAccount.Support_Tier__c` to pick the right queue DeveloperName.

**Then** — a second Get Records:

| Property          | Value                                                            |
|-------------------|------------------------------------------------------------------|
| API Name          | getQueueGroup                                                    |
| Object            | Group                                                            |
| Filter            | Type = 'Queue' AND DeveloperName = {!resolvedQueueDeveloperName} |
| How many records  | First record                                                     |
| Store in variable | varQueue                                                         |

`varQueue.Id` is the Queue Id to set as `Case.OwnerId`.

#### [4] Decision — Which Tier?

Uses `{!varAccount.Support_Tier__c}` :

| Outcome            | Condition                                                | Branch |
|--------------------|----------------------------------------------------------|--------|
| Hypercare          | <code>{!varAccount.Support_Tier__c} = "HYPERCARE"</code> | 5a     |
| Strategic          | <code>{!varAccount.Support_Tier__c} = "STRATEGIC"</code> | 5b     |
| Premium            | <code>{!varAccount.Support_Tier__c} = "PREMIUM"</code>   | 5c     |
| Standard (default) | —                                                        | 5d     |

#### [5a] HYPERCARE Branch — Assignment

Assign to `{!$Record}` :

| Field                      | Value                                               |
|----------------------------|-----------------------------------------------------|
| Account_Revenue_Tier__c    | "HYPERCARE"                                         |
| Account_RAG_at_Creation__c | <code>{!varAccount.RAG__c}</code>                   |
| Account_CSM_Name__c        | <code>{!varAccount.Account_CSM__r.Name}</code>      |
| Is_Top_Customer__c         | <code>{!varAccount.Top_Customer__c}</code>          |
| Priority                   | "Critical" (override if currently Low/Medium/Minor) |

|          |                                                  |
|----------|--------------------------------------------------|
| Queue__c | "L3-Escalation" (picklist value — for reporting) |
| OwnerId  | {!varQueue.Id} (Hypercare Support Queue)         |

#### [5b] STRATEGIC Branch — Assignment

| Field                      | Value                                                   |
|----------------------------|---------------------------------------------------------|
| Account_Revenue_Tier__c    | "STRATEGIC"                                             |
| Account_RAG_at_Creation__c | {!varAccount.RAG__c}                                    |
| Account_CSM_Name__c        | {!varAccount.Account_CSM__r.Name}                       |
| Is_Top_Customer__c         | {!varAccount.Top_Customer__c}                           |
| Priority                   | Upgrade to "Major" if currently Minor; else leave as-is |
| Queue__c                   | "L2"                                                    |
| OwnerId                    | {!varQueue.Id} (Strategic Support Queue)                |

#### [5c] PREMIUM Branch — Assignment

| Field                      | Value                                                       |
|----------------------------|-------------------------------------------------------------|
| Account_Revenue_Tier__c    | "PREMIUM"                                                   |
| Account_RAG_at_Creation__c | {!varAccount.RAG__c}                                        |
| Account_CSM_Name__c        | {!varAccount.Account_CSM__r.Name}                           |
| Is_Top_Customer__c         | {!varAccount.Top_Customer__c}                               |
| Queue__c                   | "L2"                                                        |
| OwnerId                    | leave as set by existing assignment rules (do NOT override) |

#### [5d] STANDARD Branch — Assignment

| Field                      | Value                                     |
|----------------------------|-------------------------------------------|
| Account_Revenue_Tier__c    | "STANDARD"                                |
| Account_RAG_at_Creation__c | {!varAccount.RAG__c}                      |
| Account_CSM_Name__c        | {!varAccount.Account_CSM__r.Name}         |
| Is_Top_Customer__c         | {!varAccount.Top_Customer__c}             |
| Queue__c                   | leave blank / unchanged                   |
| OwnerId                    | leave as set by existing assignment rules |

## [6] Optional — Revenue Rollup

Only if `Account_LTM_Revenue__c` is in scope (see [01](#)).

Get Records:

| Property | Value                                                                                         |
|----------|-----------------------------------------------------------------------------------------------|
| Object   | <code>accounts_revenue__c</code>                                                              |
| Filter   | <code>Account__c = {!\$Record.AccountId} AND Revenue_Booked_On__c &gt;= {!TODAY - 365}</code> |
| Fields   | <code>Revenue_Booked_Amount__c</code>                                                         |
| Store in | <code>varRevenueRecords</code> (collection)                                                   |

Then a **Loop** + Assignment accumulator computes `varTotalRevenue`, which is stamped on `Case.Account_LTM_Revenue__c`.

**Better alternative:** Build `Account.LTM_Revenue__c` as a roll-up summary (or nightly batch) and have the Flow read it from `varAccount` — avoids the second SOQL per Case. See [01 §A.8](#).

## 4. Handling Hypercare Flag Drift

Source of truth ambiguity: `Hypercare_active__c` (picklist) and `Hypercare_End_Date__c` (datetime) can diverge. Two fields, one intent.

### Option 1 — Formula on Account handles it (RECOMMENDED)

The `Support_Tier__c` formula in [01 §A.1](#) already checks both fields. The Flow just reads the formula result — no logic in Flow. **Single source of truth: the formula.**

### Option 2 — Cleanup Batch

Nightly batch job clears `Hypercare_active__c` where `Hypercare_End_Date__c < NOW()`:

```
// Scheduled Apex – runs nightly
List<Account> toUpdate = [
    SELECT Id, Hypercare_active__c
    FROM Account
    WHERE Hypercare_active__c != NULL
        AND Hypercare_End_Date__c != NULL
        AND Hypercare_End_Date__c < :DateTime.now()
        AND IsDeleted = FALSE
    LIMIT 10000
];
for (Account a : toUpdate) a.Hypercare_active__c = null;
if (!toUpdate.isEmpty()) update toUpdate;
```

Option 1 is simpler. Use unless the business needs the flag kept for audit.

## 5. Error Handling & Fault Paths

| Scenario                                                     | Handling                                                            |
|--------------------------------------------------------------|---------------------------------------------------------------------|
| Account has no Support_Tier__c (formula returns blank)       | Decision default branch → tier = STANDARD, owner unchanged          |
| Case.AccountId is null (email-to-case without Account match) | Entry condition can exclude AccountId = NULL, OR set tier = UNKNOWN |
| Queue_Config__mdt has no matching tier record                | Log debug, set tier but don't change OwnerId                        |
| Queue DeveloperName doesn't resolve to a Group record        | Log debug, leave OwnerId unchanged                                  |
| Existing OwnerId is a user (not queue)                       | Override for HYPERCARE/STRATEGIC only — Flow does this by design    |

**Flow Fault Paths:** Add fault connectors on Get Records elements pointing to an error-logging subflow or Platform Event publish. Never let the Flow fail silently and block Case creation.

## 6. Interaction with Existing Case Assignment Rules

Salesforce processes trigger actions in this order:

1. Before Save Flows (this Flow runs here)
2. Validation rules
3. Save record
4. After Save Flows / Process Builder
5. Assignment rules (if `AssignmentRuleHeader` set on DML) or default owner

**Implication:** If this Flow sets `OwnerId` to a Queue, then the assignment rule runs after — and **may override the OwnerId set by this Flow**.

### Remediation options:

**Option A — Disable assignment rules for in-scope record types** Modify the assignment rule entries to skip records where `Account_Revenue_Tier__c IN ( 'HYPERCARE', 'STRATEGIC' )`.

**Option B — Use After-Save flow instead** Move OwnerId changes to an After-Save Flow that fires after assignment rules. Slower but avoids conflict. Field stamps stay in Before-Save.

**Option C — Use Apex trigger with `Database.DMLOptions` disabling assignment rules for this subset** Most control, highest complexity.

**Recommendation:** Start with Option A for HYPERCARE/STRATEGIC only. These are the tiers where we absolutely want to override normal routing. PREMIUM lets assignment rules run normally.

## 7. Governor Limit Analysis

Per Case creation:

- **SOQL queries:** 2 (Account, Queue\_Config) + 1 (Group) + 1 optional (accounts\_revenue\_\_c) = **3–4 SOQL/Case**
- **DML:** 0 (Before Save stamps are free)

- **CPU time:** ~10ms expected

Salesforce Flow governor limits:

- SOQL per transaction: 100 → we use ~4 → safe by 25x
- CPU time: 10,000ms → we use ~10 → safe by 1,000x

**At 500 Cases/day, this is negligible. At 10,000 Cases/day (bulk email-to-case surge), still safe.**

---

## 8. Testing Strategy

### 8.1 — Unit Tests (Apex Test Class)

```
@isTest
public class Case_RevenueTier_FlowTest {

    @testSetup
    static void setup() {
        // Create test users, accounts with each tier combo
        User csm = new User(/* ... */);
        insert csm;

        Account hypercare = new Account(
            Name = 'Test Hypercare',
            Top_Customer__c = false,
            Hypercare_active__c = 'Active',
            Hypercare_End_Date__c = DateTime.now().addDays(30),
            Account_CSM__c = csm.Id
        );
        Account strategic = new Account(
            Name = 'Test Strategic',
            Top_Customer__c = true,
            Account_CSM__c = csm.Id
        );
        Account premium = new Account(
            Name = 'Test Premium',
            SAAS_Enterprise_Account__c = true,
            RAG__c = 'GREEN',
            Account_CSM__c = csm.Id
        );
        Account standard = new Account(
            Name = 'Test Standard',
            Account_CSM__c = csm.Id
        );
        insert new List<Account>{ hypercare, strategic, premium, standard };
    }

    @isTest
    static void testHypercareTierStamped() {
        Account a = [SELECT Id FROM Account WHERE Name = 'Test Hypercare' LIMIT 1];
        Case c = new Case(AccountId = a.Id, Subject = 'Test', Priority = 'Minor',
            Status = 'New');
```



```

        insert c;

        Case stamped = [SELECT Account_Revenue_Tier__c, Priority, OwnerId FROM Case
WHERE Id = :c.Id];
        System.assertEquals('HYPERCARE', stamped.Account_Revenue_Tier__c);
        System.assertEquals('Critical', stamped.Priority, 'Hypercare should upgrade
priority');
        // Further asserts on OwnerId type (Queue)
    }

    @isTest
    static void testStrategicTierStamped() {
        Account a = [SELECT Id FROM Account WHERE Name = 'Test Strategic' LIMIT 1];
        Case c = new Case(AccountId = a.Id, Subject = 'Test', Priority = 'Minor',
Status = 'New');
        insert c;
        Case stamped = [SELECT Account_Revenue_Tier__c FROM Case WHERE Id = :c.Id];
        System.assertEquals('STRATEGIC', stamped.Account_Revenue_Tier__c);
    }

    @isTest
    static void testPremiumTierStamped() {
        Account a = [SELECT Id FROM Account WHERE Name = 'Test Premium' LIMIT 1];
        Case c = new Case(AccountId = a.Id, Subject = 'Test', Status = 'New');
        insert c;
        Case stamped = [SELECT Account_Revenue_Tier__c FROM Case WHERE Id = :c.Id];
        System.assertEquals('PREMIUM', stamped.Account_Revenue_Tier__c);
    }

    @isTest
    static void testStandardTierStamped() {
        Account a = [SELECT Id FROM Account WHERE Name = 'Test Standard' LIMIT 1];
        Case c = new Case(AccountId = a.Id, Subject = 'Test', Status = 'New');
        insert c;
        Case stamped = [SELECT Account_Revenue_Tier__c FROM Case WHERE Id = :c.Id];
        System.assertEquals('STANDARD', stamped.Account_Revenue_Tier__c);
    }

    @isTest
    static void testNoAccountHandlesGracefully() {
        Case c = new Case(Subject = 'No Account', Status = 'New');
        insert c;
        Case stamped = [SELECT Account_Revenue_Tier__c FROM Case WHERE Id = :c.Id];
        System.assertEquals('UNKNOWN', stamped.Account_Revenue_Tier__c);
    }

    @isTest
    static void testHypercareExpiredFallsBackToNormal() {
        Account a = [SELECT Id FROM Account WHERE Name = 'Test Hypercare' LIMIT 1];
        a.Hypercare_End_Date__c = DateTime.now().addDays(-5); // expired
        update a;
        Case c = new Case(AccountId = a.Id, Subject = 'Test', Status = 'New');

```

```

        insert c;
        Case stamped = [SELECT Account_Revenue_Tier__c FROM Case WHERE Id = :c.Id];
        System.assertNotEquals('HYPERCARE', stamped.Account_Revenue_Tier__c,
            'Expired Hypercare should fall back');
    }

    @isTest
    static void testBulkInsert200Cases() {
        List<Account> accounts = [SELECT Id FROM Account LIMIT 4];
        List<Case> cases = new List<Case>();
        for (Integer i = 0; i < 200; i++) {
            cases.add(new Case(
                AccountId = accounts[Math.mod(i, 4)].Id,
                Subject = 'Bulk Test ' + i,
                Status = 'New'
            ));
        }
        Test.startTest();
        insert cases;
        Test.stopTest();

        Integer stamped = [SELECT COUNT() FROM Case WHERE Account_Revenue_Tier__c !=
            null AND Id IN :cases];
        System.assertEquals(200, stamped, 'All 200 bulk cases should have tier
            stamped');
    }
}

```

## 8.2 — UAT Scenarios

See [06 — Test & Rollout](#) for the full UAT matrix.

## 9. Deployment Checklist

- ☐ New Case fields deployed ([01 §A.3–A.6](#))
- ☐ Account.Support\_Tier\_\_c formula deployed ([01 §A.1](#))
- ☐ Queue\_Config\_\_mdt Custom Metadata Type deployed with all 4 tier records
- ☐ 4 Salesforce Queues created: Hypercare\_Support\_Queue, Strategic\_Support\_Queue, Premium\_Support\_Queue, FLR\_Support\_Queue (last may already exist)
- ☐ Queue membership: active L2+ agents assigned to Strategic/Hypercare/Premium queues
- ☐ Flow deployed to sandbox, tested with scenarios above
- ☐ Existing Case assignment rules reviewed — add skip condition for HYPERCARE/STRATEGIC tiers
- ☐ Apex test class passing with > 90% coverage
- ☐ Deploy to production via change set or SFDX package

## 10. Flow XML Skeleton (SFDX)

Path: `force-app/main/default/flows/Case_RevenueTier_BeforeSave.flow-meta.xml`

```

<?xml version="1.0" encoding="UTF-8"?>
<Flow xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>60.0</apiVersion>
  <description>Stamps revenue tier and CSM on Case at creation. See docs/revenue-
in-support/02-case-flow-specification.md</description>
  <environments>Default</environments>
  <interviewLabel>Case Revenue Tier {!$Flow.CurrentDateTime}</interviewLabel>
  <label>Case – Revenue Tier Enrichment (Before Save)</label>
  <processMetadataValues>
    <name>BuilderType</name>
    <value><stringValue>LightningFlowBuilder</stringValue></value>
  </processMetadataValues>
  <processType>AutoLaunchedFlow</processType>
  <runInMode>SystemModeWithSharing</runInMode>
  <start>
    <locationX>50</locationX>
    <locationY>0</locationY>
    <connector>
      <targetReference>getAccount</targetReference>
    </connector>
    <object>Case</object>
    <recordTriggerType>Create</recordTriggerType>
    <triggerType>RecordBeforeSave</triggerType>
  </start>
  <status>Draft</status>

  <!-- =====
    Get Records: Account
    ===== -->
  <recordLookups>
    <name>getAccount</name>
    <label>Get Account</label>
    <locationX>50</locationX>
    <locationY>100</locationY>
    <assignNullValuesIfNoRecordsFound>true</assignNullValuesIfNoRecordsFound>
    <connector>
      <targetReference>decideTier</targetReference>
    </connector>
    <filterLogic>and</filterLogic>
    <filters>
      <field>Id</field>
      <operator>EqualTo</operator>
      <value>
        <elementReference>$Record.AccountId</elementReference>
      </value>
    </filters>
    <getFirstRecordOnly>true</getFirstRecordOnly>
    <object>Account</object>
    <outputReference>varAccount</outputReference>
    <queriedFields>Id</queriedFields>
    <queriedFields>Name</queriedFields>
  </recordLookups>
</Flow>

```

```

    <queriedFields>Top_Customer__c</queriedFields>
    <queriedFields>Is_SAAS_Premium_Account__c</queriedFields>
    <queriedFields>SAAS_Enterprise_Account__c</queriedFields>
    <queriedFields>RAG__c</queriedFields>
    <queriedFields>Support_Tier__c</queriedFields>
    <queriedFields>Hypercare_active__c</queriedFields>
    <queriedFields>Hypercare_End_Date__c</queriedFields>
    <queriedFields>Account_CSM__c</queriedFields>
    <storeOutputAutomatically>false</storeOutputAutomatically>
</recordLookups>

<!-- =====
      Decision: Tier
      ===== -->

<decisions>
  <name>decideTier</name>
  <label>Which Tier?</label>
  <locationX>50</locationX>
  <locationY>200</locationY>
  <defaultConnector>
    <targetReference>assignStandard</targetReference>
  </defaultConnector>
  <defaultConnectorLabel>Standard</defaultConnectorLabel>
  <rules>
    <name>isHypercare</name>
    <conditionLogic>and</conditionLogic>
    <conditions>
      <leftValueReference>varAccount.Support_Tier__c</leftValueReference>
      <operator>EqualTo</operator>
      <rightValue><stringValue>HYPERCARE</stringValue></rightValue>
    </conditions>
    <connector>
      <targetReference>assignHypercare</targetReference>
    </connector>
    <label>Hypercare</label>
  </rules>
  <rules>
    <name>isStrategic</name>
    <conditionLogic>and</conditionLogic>
    <conditions>
      <leftValueReference>varAccount.Support_Tier__c</leftValueReference>
      <operator>EqualTo</operator>
      <rightValue><stringValue>STRATEGIC</stringValue></rightValue>
    </conditions>
    <connector>
      <targetReference>assignStrategic</targetReference>
    </connector>
    <label>Strategic</label>
  </rules>
  <rules>
    <name>isPremium</name>
    <conditionLogic>and</conditionLogic>

```

```

        <conditions>
            <leftValueReference>varAccount.Support_Tier__c</leftValueReference>
            <operator>EqualTo</operator>
            <rightValue><stringValue>PREMIUM</stringValue></rightValue>
        </conditions>
        <connector>
            <targetReference>assignPremium</targetReference>
        </connector>
        <label>Premium</label>
    </rules>
</decisions>

<!-- NOTE: assignHypercare, assignStrategic, assignPremium, assignStandard
elements not shown in this skeleton – each is a <assignments> element
that sets fields on $Record per section 3 of this doc.
The Queue resolution (Get Records on Queue_Config__mdt + Group)
should be inline before each assignment, OR done once and branched. -->

<variables>
    <name>varAccount</name>
    <dataType>SObject</dataType>
    <isCollection>false</isCollection>
    <isInput>false</isInput>
    <isOutput>false</isOutput>
    <objectType>Account</objectType>
</variables>
</Flow>

```

This is a **skeleton**, not a complete deployment. The assignment elements and Queue resolution logic should be completed in Flow Builder (faster and less error-prone than hand-editing XML). Use this skeleton as the structural starting point.